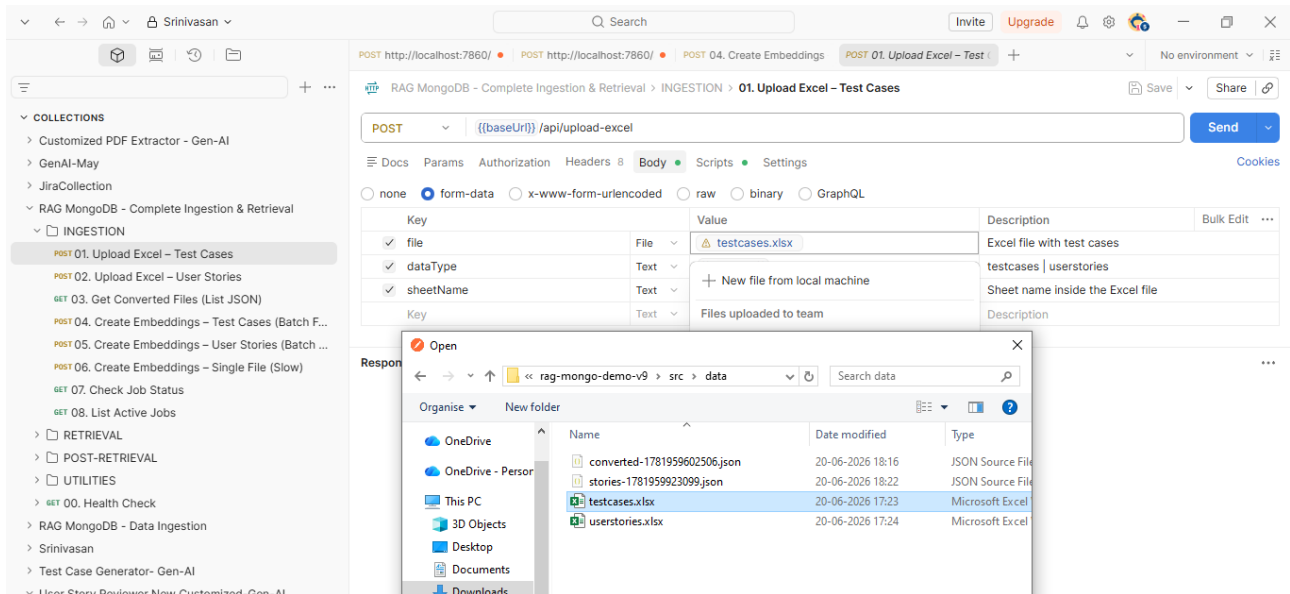
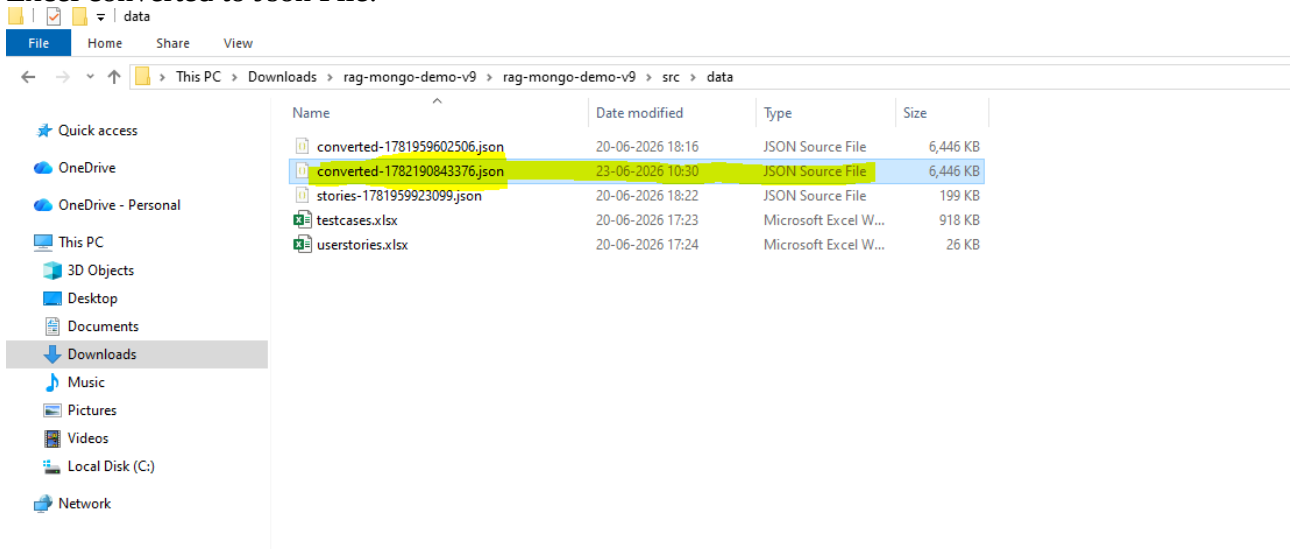


Ingestion:

01.Upload Excel – Test Cases



Excel converted to Json File:



02. Upload Excel – User Stories: Uploading user stories file

POST 02. Upload Excel – User Stories

POST `[[baseUrl]]/api/upload-excel`

Body

Key	Value	Description	Bulk Edit
file	File	userstories.xlsx	Excel file with user stories
dataType	Text	userstories	
sheetName	Text	stories	Sheet name inside the Excel file
Key	Text	Value	Description

Body

```
{
  "success": true,
  "message": "User stories file converted successfully",
  "outputFile": "stories-1782191748899.json",
  "output": "Converting 198 rows from sheet \"stories\"...nConverted 198 user stories from \"stories\" into C:/Users/Srinivasan/Downloads/rag-mongo-demo-v9/src/data/stories-1782191748899.jsonnFiltered out: 0 empty rowsn"
}
```

User stories converted to Json:

File Explorer

Path: This PC > Downloads > rag-mongo-demo-v9 > rag-mongo-demo-v9 > src > data

Name	Date modified	Type	Size
converted-1781959602506.json	20-06-2026 18:16	JSON Source File	6,446 KB
converted-1782190843376.json	23-06-2026 10:30	JSON Source File	6,446 KB
stories-178195923099.json	20-06-2026 18:22	JSON Source File	199 KB
stories-1782191748899.json	23-06-2026 10:45	JSON Source File	199 KB
testcases.xlsx	20-06-2026 17:23	Microsoft Excel W...	918 KB
userstories.xlsx	20-06-2026 17:24	Microsoft Excel W...	26 KB

03. Get Converted Files(List JSON), retrieves data from the src folder

GET 03. Get Converted Files (List JSON)

GET `[[baseUrl]]/api/files`

Headers

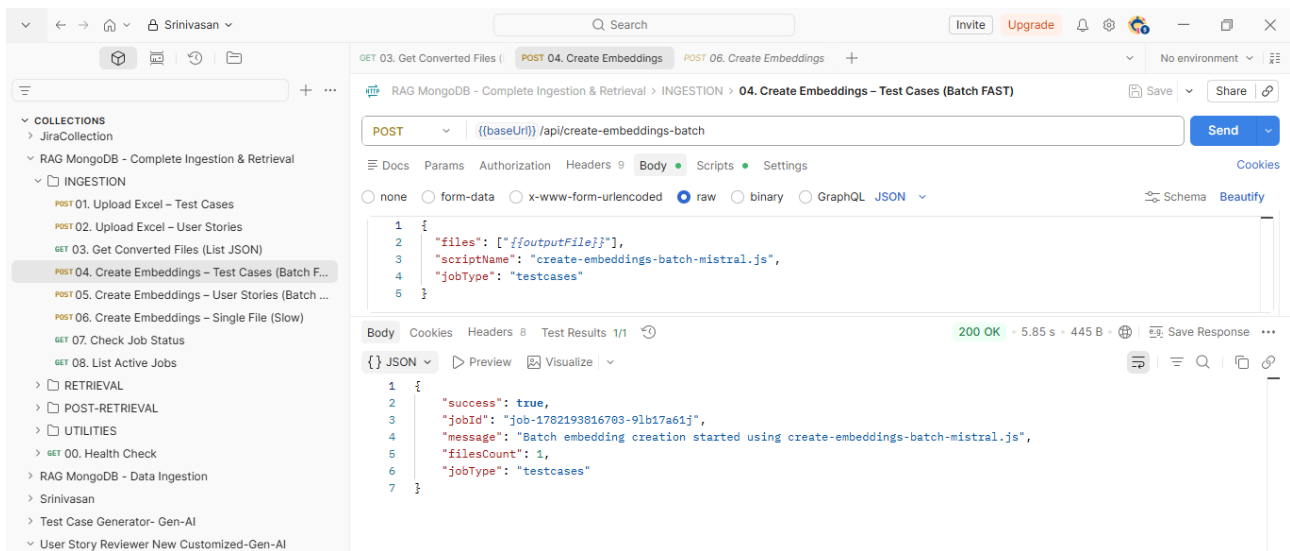
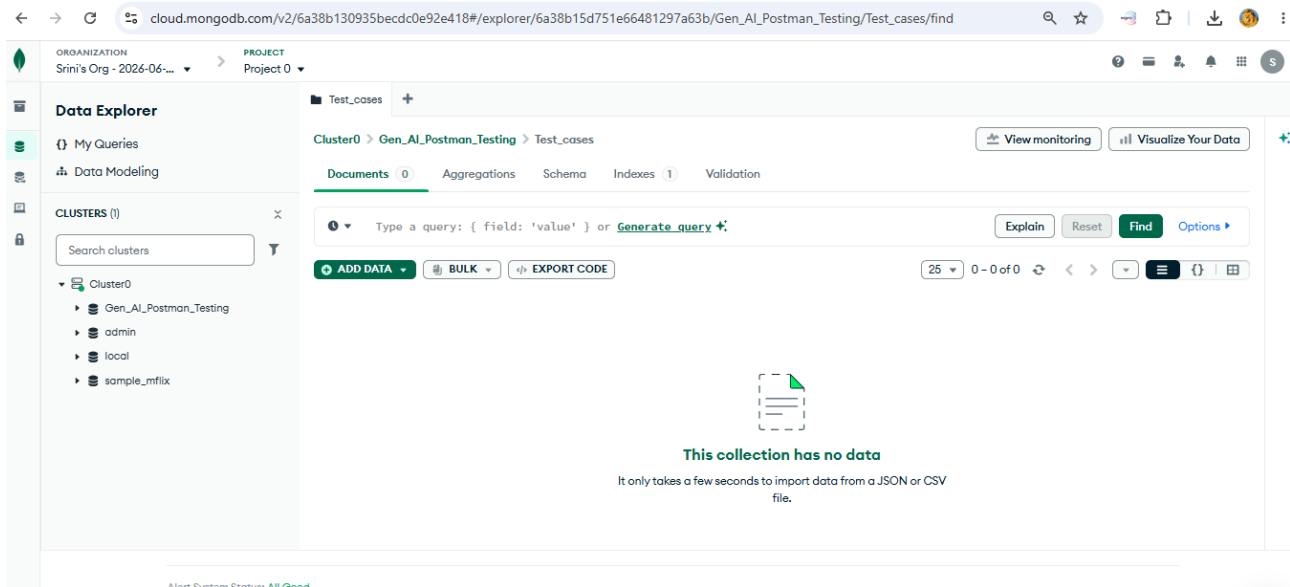
Header	Value
Postman-Token	<calculated when request is sent>
Host	<calculated when request is sent>
User-Agent	PostmanRuntime/7.5.0

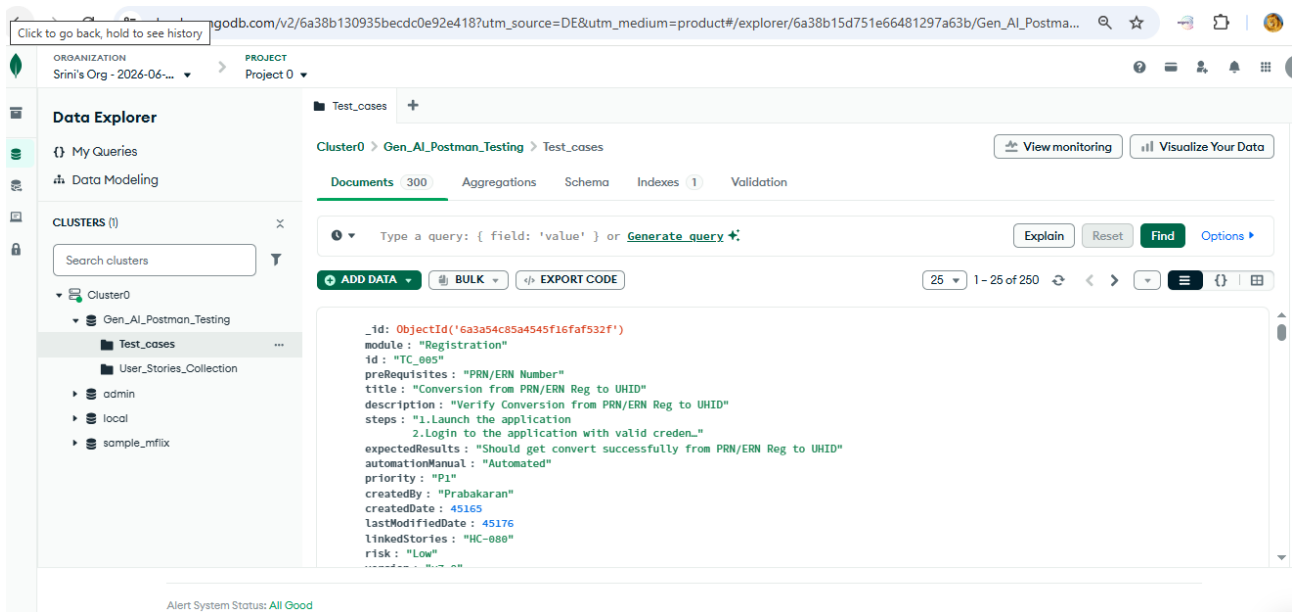
Body

```
{
  "name": "converted-1782190843376.json",
  "path": "C:\\Users\\Srinivasan\\Downloads\\rag-mongo-demo-v9\\rag-mongo-demo-v9\\src\\data\\converted-1782190843376.json",
  "size": 6608526,
  "modified": "2026-06-23T05:00:47.327Z",
  "type": "json"
},
{
  "name": "stories-178195923099.json",
  "path": "C:\\Users\\Srinivasan\\Downloads\\rag-mongo-demo-v9\\rag-mongo-demo-v9\\src\\data\\stories-178195923099.json",
  "size": 203046,
  "modified": "2026-06-20T12:52:03.778Z",
  "type": "json"
},
{
  "name": "stories-1782191748899.json",
  "path": "C:\\Users\\Srinivasan\\Downloads\\rag-mongo-demo-v9\\rag-mongo-demo-v9\\src\\data\\stories-1782191748899.json",
  "size": 203046,
  "modified": "2026-06-23T05:18:44.591Z",
  "type": "json"
}
```

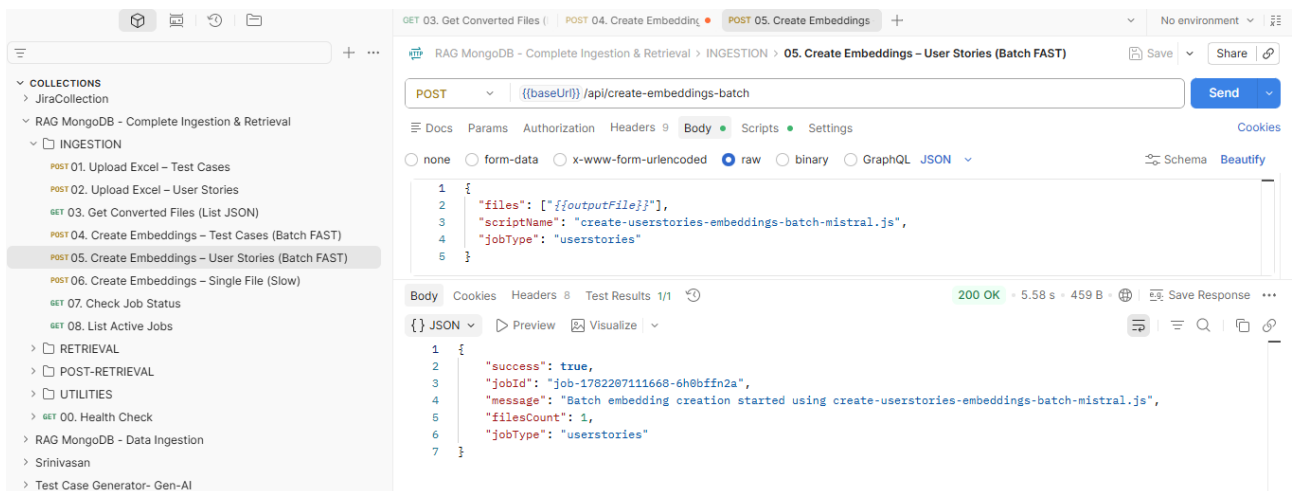
04. Create Embeddings – Test Cases (Batch FAST):

Before ingestion and embeddings:

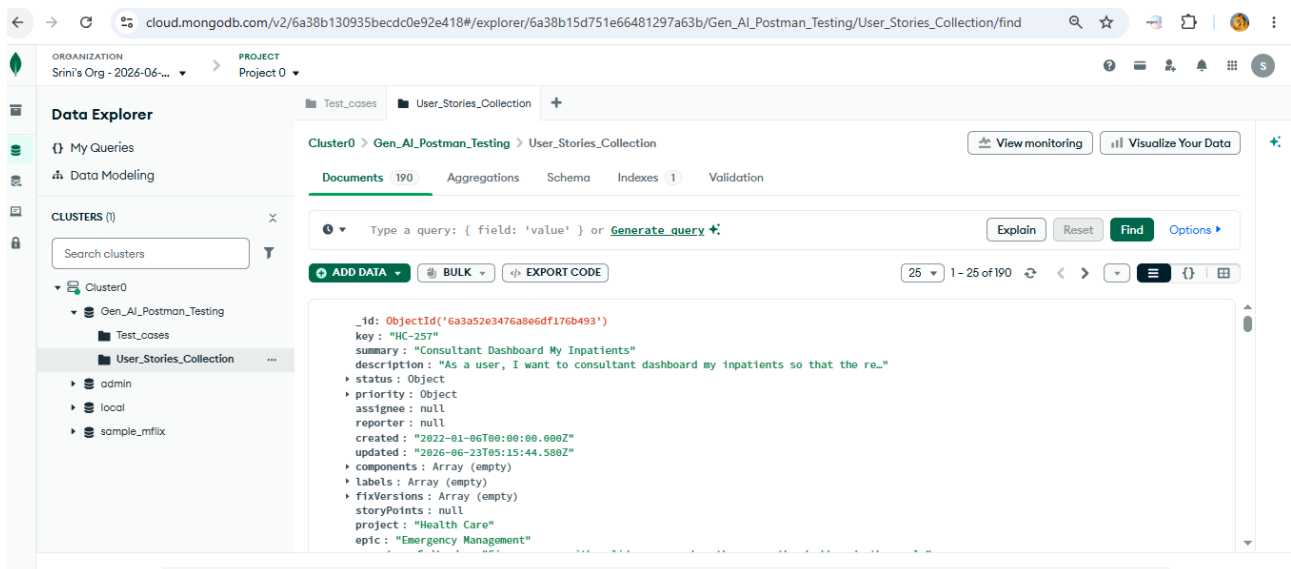




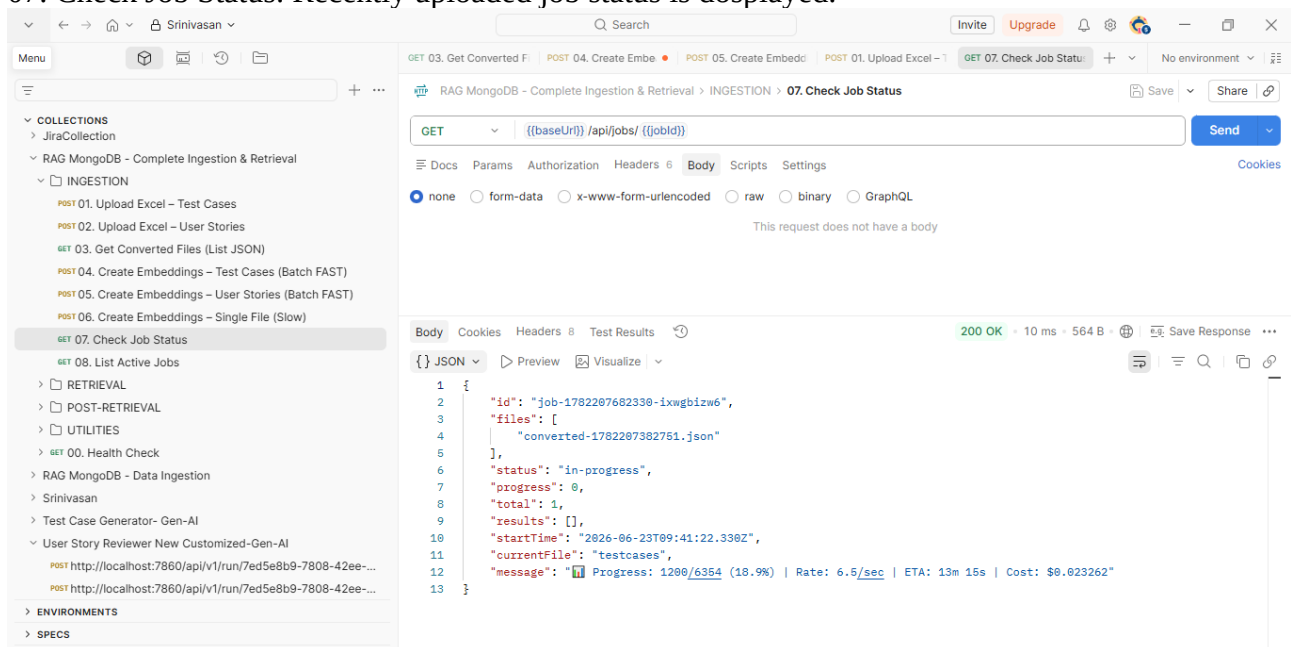
05. Create Embeddings – User Stories (Batch FAST)



User stories embedded and ingestion done in vector DB.



07. Check Job Status: Recently uploaded job status is displayed.



08. List Active Jobs: Active job status is displayed here

Screenshot of a REST client interface showing a GET request to `/api/jobs/active` returning a 200 OK response. The response body is a JSON array containing one job object.

```
GET 03. Get Converted Files (List JSON) POST 04. Create Embeddings - Test Cases POST 05. Create Embeddings - User Stories (Batch FAST) GET 08. List Active Jobs GET 07. Check Job Status
```

Body: `{ "jobs": [{ "id": "job-1782207682330-ixwgbizw6", "files": ["converted-1782207382751.json"], "status": "in-progress", "progress": 0, "total": 1, "results": [], "startTime": "2026-06-23T09:41:22.338Z", "currentFile": "testcases", "message": "Progress: 2100/6354 (33.1%) | Rate: 6.3/sec | ETA: 11m 28s | Cost: $0.039537" }] }`

Retrieval:

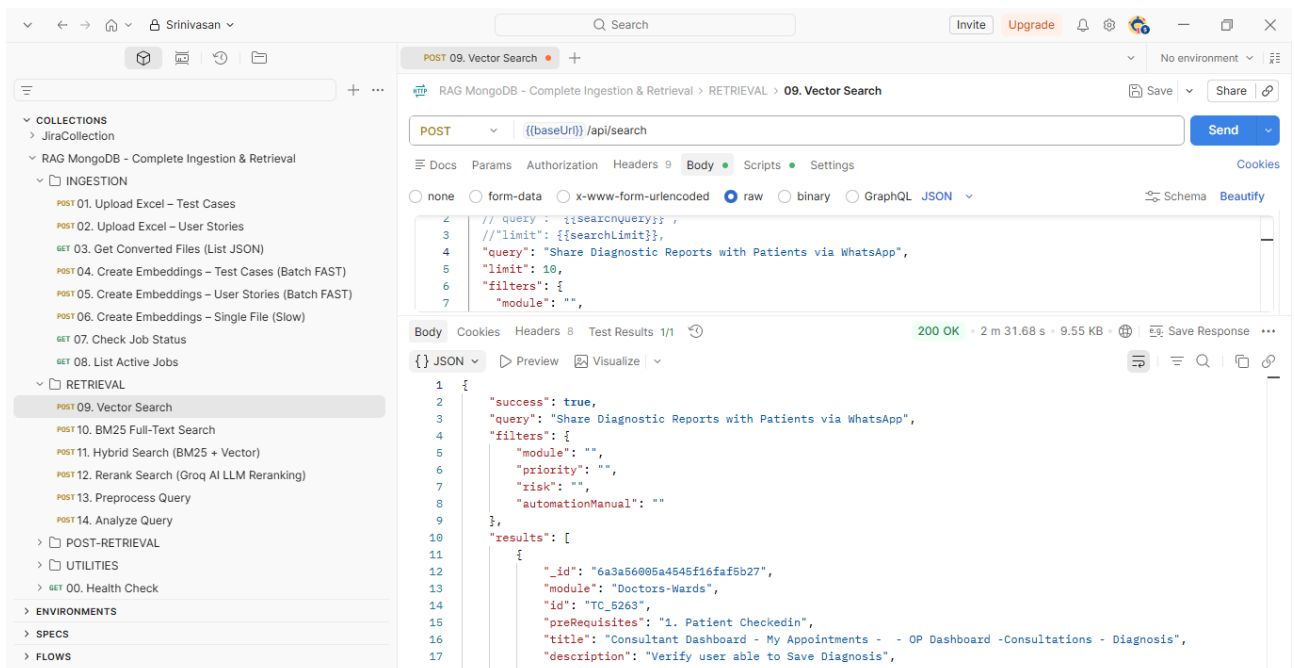
09. Vector Search

Screenshot of a REST client interface showing a POST request to `/api/search` returning a 200 OK response. The request body is a JSON object with search parameters, and the response body is a JSON object with search results.

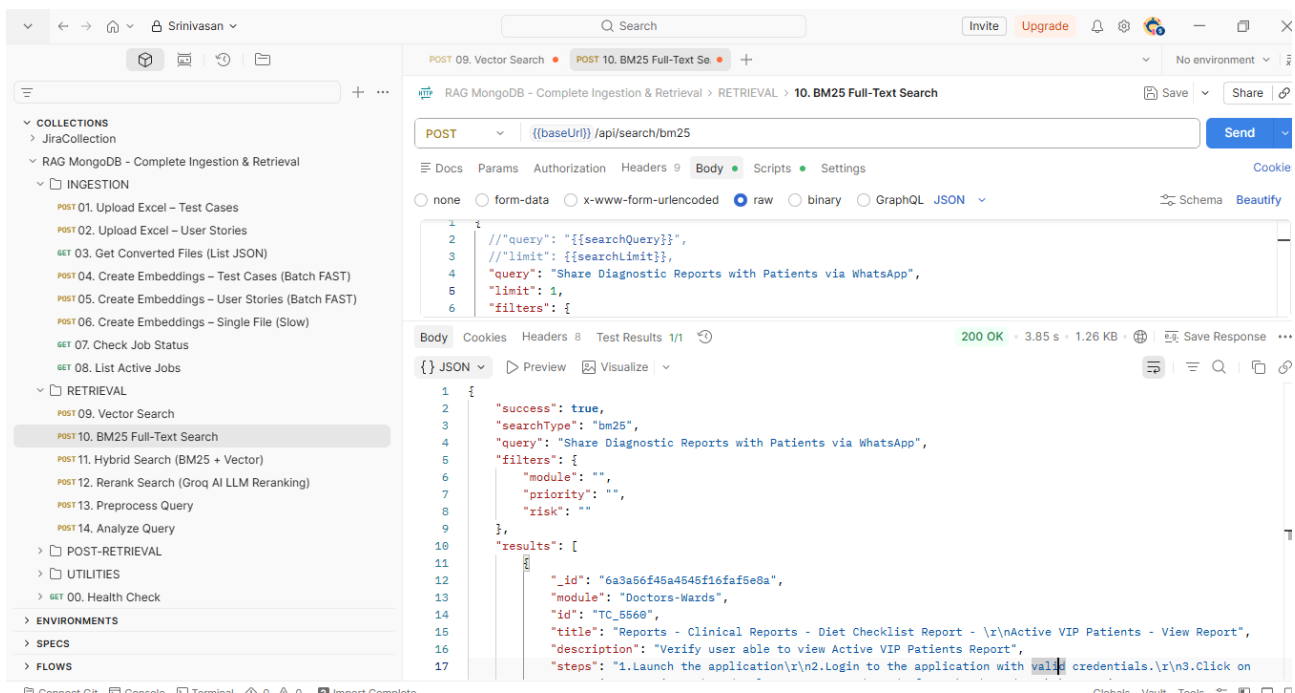
```
POST 09. Vector Search
```

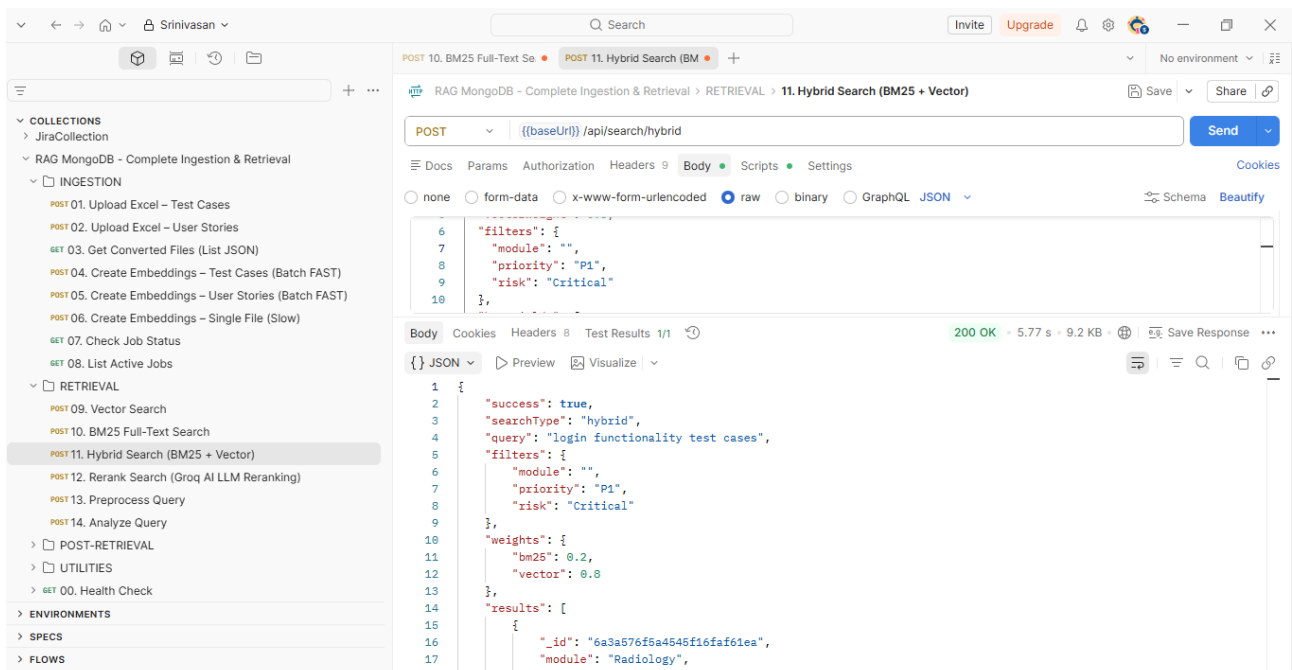
Body: `{ "query": "{{searchQuery}}", "limit": "{{searchLimit}}", "filters": { "module": "", "priority": "", "risk": "", "automationManual": "" } }`

Body: `{ "success": true, "query": "login functionality test cases", "filters": { "module": "", "priority": "", "risk": "", "automationManual": "" }, "results": [{ "_id": "6a3a576f5a4545f16faf61ea", "module": "Radiology", "id": "TC_3701", "preRequisites": "Valid Employee ID\\r\\n(Employee ID-103171)", "title": "Radiology-Masters-Login Modality Master-Save", "description": "Verify user is able to Save Login Modality Master" }] }`

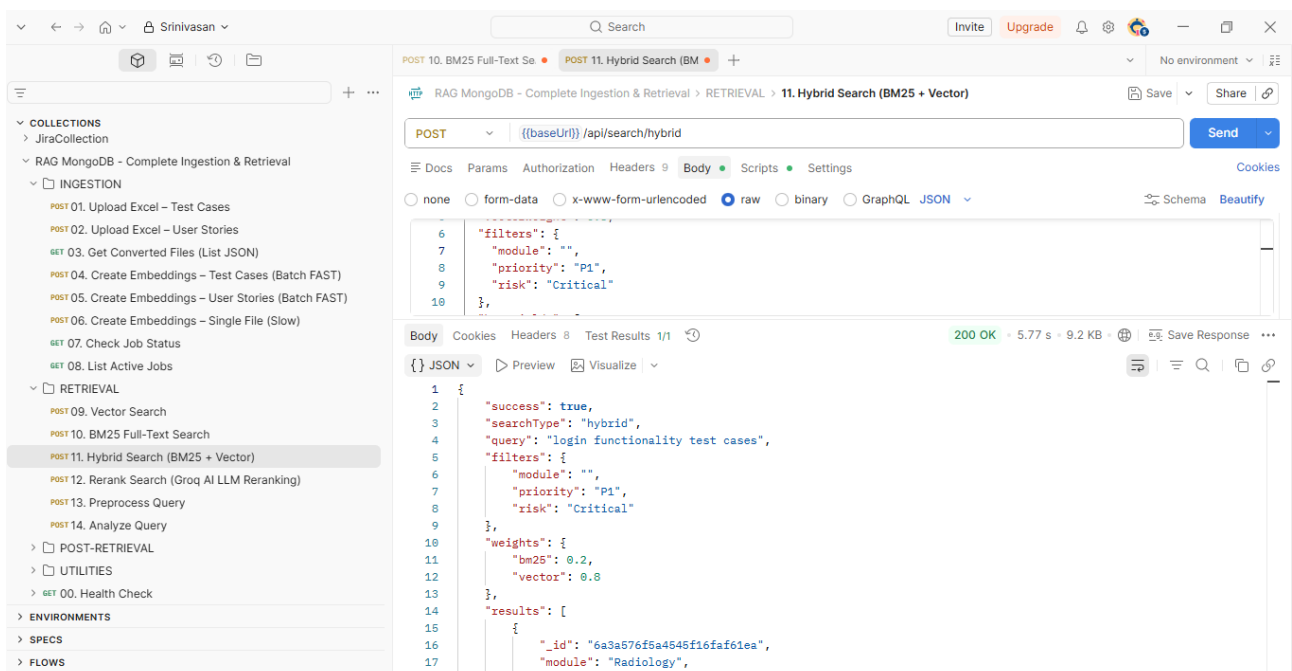


10. BM25 Full-Text Search: searching BM25 index with the query and limit as 1:

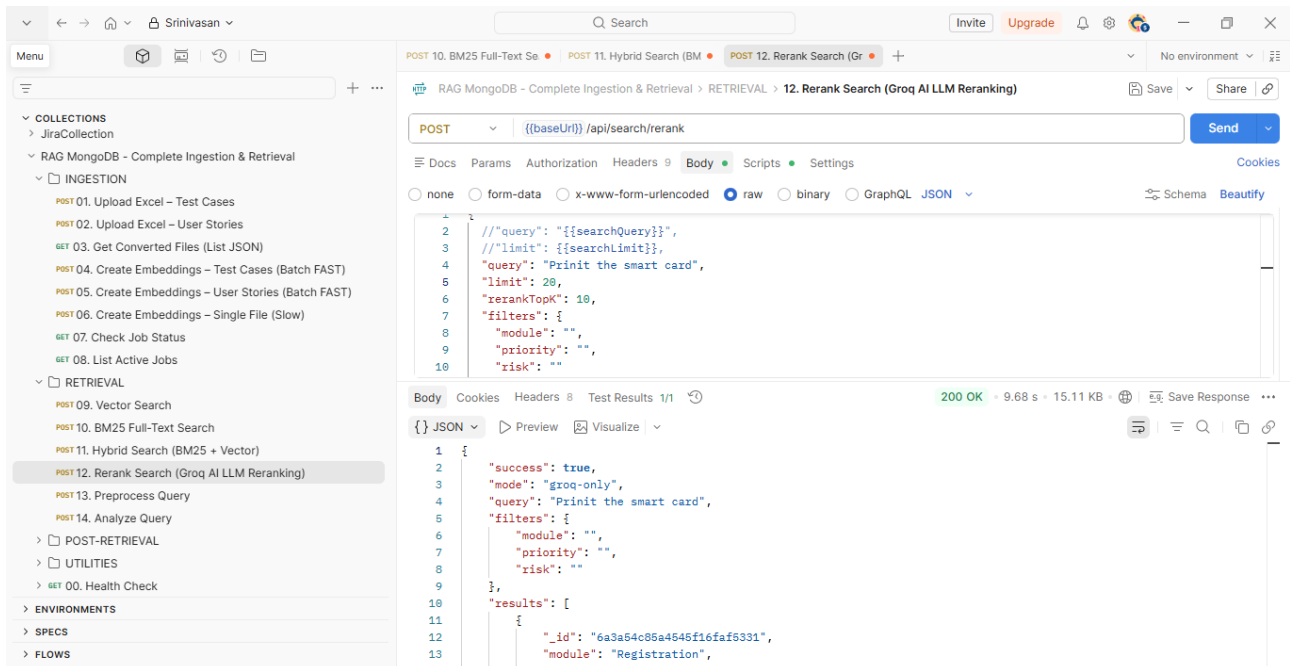




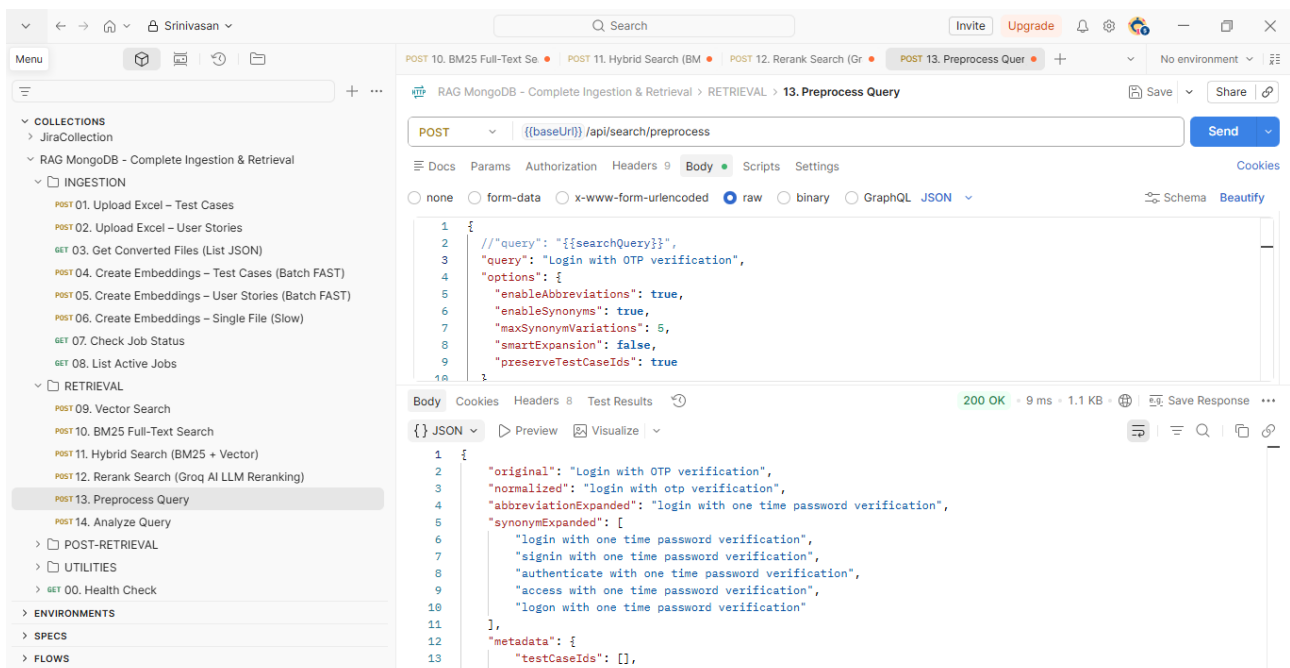
11. Hybrid Search (BM25 + Vector): Changing Priority and risk-based



12. Rerank Search (Groq AI LLM Reranking):



13. Preprocess Query: Login with OTP verification. Cleans, expands, and optimizes the user's input before sending it to the retrieval models.



14. Analyze Query: Interprets the processed query to understand **intent**, **context**, and **expected output type**.

POST 10. BM25 Full-Text Search POST 11. Hybrid Search (BM25 + Vector) POST 12. Rerank Search (Groq AI LLM Reranking) POST 13. Preprocess Query POST 14. Analyze Query POST 15. Summarize Results (Concise) POST 16. Summarize Results (Detailed) POST 17. Deduplicate Results POST 18. Test AI Prompt (Groq)

POST 19. Get Metadata (Distinct Filter Values) GET 20. Get Latest Test Case ID GET 21. Get Environment Variables POST 22. Update Environment Variables GET 00. Health Check RAG MongoDB - Data Ingestion Srinivasan Test Case Generator- Gen-AI User Story Reviewer New Customized-Gen-AI POST http://localhost:7860/api/v1/run/7ed5e8b9-7808-42ee-...

ENVIRONMENTS SPECS FLOWS

POST 14. Analyze Query

POST ((baseUrl)) /api/search/analyze

Body Params Authorization Headers 9 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   // "query": "{{searchQuery}}"
3   "query": "Login with OTP verification"
4 }
```

Body Cookies Headers 8 Test Results Save Response

JSON Preview Visualize

```
1 {
2   "original": "Login with OTP verification",
3   "normalized": "login with otp verification",
4   "analysis": {
5     "hasTestCaseIds": false,
6     "testCaseIds": [],
7     "hasAbbreviations": true,
8     "abbreviations": [
9       {
10        "abbreviation": "otp",
11        "expansion": "one time password",
12        "position": 11
13      }
14    ],
15     "hasSynonymOpportunities": true,
16     "synonymOpportunities": [
17       {
18        "text": "login",
```

Post-Retrieval: 15. Summarize Results (Concise)

POST 13. Preprocess Query POST 14. Analyze Query POST 15. Summarize Results (Concise) POST 16. Summarize Results (Detailed)

POST ((baseUrl)) /api/search/summarize

Body Params Authorization Headers 9 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   // "query": "{{searchQuery}}",
3   "query": "Login with OTP verification",
4   "summaryType": "concise",
5   "results": [
6     {
7       "success": true,
8       "summary": "**Search Results Overview (1 result)** \n\n ID | Title | Module | Key Details |\n-----|-----|-----|-----|
9       TC_0001 | Verify valid user login | Login | This test case checks that a
10      legitimate user can successfully log in to the system. It validates the standard login flow but does not
11      explicitly mention OTP (One-Time Password) verification. \n\n**Takeaway:** The sole result pertains to a
12      basic login validation test. While it confirms that a valid user can access the system, it does not address
13      OTP-based authentication, which may require additional test cases or documentation.",
14       "summaryType": "concise",
15       "resultCount": 1,
16       "model": "openai/gpt-oss-120b",
17       "tokens": {
18         "prompt": 104,
19         "completion": 149,
20         "total": 253
21       }
22     }
23   ]
24 }
```

16. Summarize Results (Detailed)

POST 13. Preprocess Quer POST 14. Analyze Query POST 15. Summarize Results POST 16. Summarize Results (Detailed) No environment

Menu

COLLECTIONS

- POST 10. BM25 Full-Text Search
- POST 11. Hybrid Search (BM25 + Vector)
- POST 12. Rerank Search (Groq AI LLM Reranking)
- POST 13. Preprocess Query
- POST 14. Analyze Query
- POST-RETRIEVAL
 - POST 15. Summarize Results (Concise)
 - POST 16. Summarize Results (Detailed)
 - POST 17. Deduplicate Results
 - POST 18. Test AI Prompt (Groq)
- UTILITIES
 - GET 19. Get Metadata (Distinct Filter Values)
 - GET 20. Get Latest Test Case ID
 - GET 21. Get Environment Variables
 - POST 22. Update Environment Variables
- ENVIRONMENTS
 - GET 00. Health Check
- RAG MongoDB - Data Ingestion
 - Srinivasan
 - Test Case Generator- Gen-AI
 - User Story Reviewer New Customized-Gen-AI
- SPECS
- FLows

POST 16. Summarize Results (Detailed)

POST `{{(baseUri)}} /api/search/summarize` Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON Schema Beautify

```
1 {
2   "query": "${searchQuery}",
3   "summaryType": "detailed",
4   "results": [
5     {
6       "id": "TC_0001"
```

Body Cookies Headers 8 Test Results Save Response

200 OK • 3.18 s • 5.63 KB

login | Login | Test that a valid user can log in successfully | 1. Quantitative Overview
Metric | Value | Total results retrieved | 1 | Results with full test-case
metadata (ID, Title, Module, Description) | 1 | Results that directly address the query "login
functionality test cases" | 1 (100% relevance) | Results lacking additional context (e.g.,
pre-conditions, expected results) | 1 (100%) | 2. Qualitative Themes | Theme |
Frequency | Interpretation | 1 | Login success scenario | 1 | The
sole result focuses on the "positive" path - confirming that a correctly-credentialed user can access the
system. | 1 | Module specificity | 1 | The test case is explicitly tied to the "Login" module, indicating
a clear functional boundary. | 1 | Minimal description | 1 | Only a brief description is provided; no
details on steps, data, or expected outcomes. | Key Takeaway: The result aligns with the most
fundamental login test case-verifying that a valid user can log in. It does not cover negative or edge-case
scenarios (e.g., invalid credentials, locked accounts, password expiration, UI validation, security checks).
3. Relevance Insights | Aspect | Assessment | Title
relevance | High - directly mentions "Verify valid user login." | Module relevance | High - the
module is explicitly "Login." | Description relevance | Moderate - confirms the purpose but lacks
depth (no steps, data, or pass/fail criteria). | Overall relevance scores (subjective 0-10) | 8/10
- The result is on-topic but limited in scope and detail. | Why the result is considered relevant: |

17. Deduplicate Results:

POST 14. Analyze Query POST 15. Summarize Results POST 16. Summarize Results POST 17. Deduplicate Results No environment

Menu

COLLECTIONS

- POST 10. BM25 Full-Text Search
- POST 11. Hybrid Search (BM25 + Vector)
- POST 12. Rerank Search (Groq AI LLM Reranking)
- POST 13. Preprocess Query
- POST 14. Analyze Query
- POST-RETRIEVAL
 - POST 15. Summarize Results (Concise)
 - POST 16. Summarize Results (Detailed)
 - POST 17. Deduplicate Results
 - POST 18. Test AI Prompt (Groq)
- UTILITIES
 - GET 19. Get Metadata (Distinct Filter Values)
 - GET 20. Get Latest Test Case ID
 - GET 21. Get Environment Variables
 - POST 22. Update Environment Variables
- ENVIRONMENTS
 - GET 00. Health Check
- RAG MongoDB - Data Ingestion
 - Srinivasan
 - Test Case Generator- Gen-AI
 - User Story Reviewer New Customized-Gen-AI
- SPECS
- FLows

POST 17. Deduplicate Results

POST `{{(baseUri)}} /api/search/deduplicate` Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON Schema Beautify

```
3 results: [
4   {
5     "id": "TC_0001",
6     "title": "Verify valid user login"
7   },
8   {
9     "id": "TC_0002",
10    "title": "Verify valid user login flow"
11  }
12 ],
13 "deduplicated": [
14   {
15     "id": "TC_0001",
16     "title": "Verify valid user login"
17   },
18   {
19     "id": "TC_0002",
20     "title": "Verify valid user login flow"
21   }
22 ],
23 ]
```

Body Cookies Headers 8 Test Results Save Response

200 OK • 7 ms • 630 B

18. Test AI Prompt (Groq)

The screenshot shows the Postman interface for a POST request to `POST 18. Test AI Prompt (Groq)`. The request body is a JSON object with the following fields:

```
{  "prompt": "Given the following test case titles, identify which ones cover edge cases:\n1. Verify valid user login\n2. Verify login with empty password\n3. Verify login with SQL injection\n4. Verify dashboard loads after login",  "temperature": 0.2,  "maxTokens": 2000}
```

The response is a JSON object with the following fields:

```
{  "success": true,  "response": {    "raw": "To identify which test case titles cover edge cases, let's first define what edge cases are. Edge cases, also known as boundary cases, are inputs or conditions that occur at the extreme ends of a range or that are unusual or exceptional in some way. Testing edge cases helps ensure that a system behaves as expected under unusual or extreme conditions.\n\nNow, let's analyze the given test case titles:\n\n1. **Verify valid user login** - This test case seems to cover a normal or happy path scenario rather than an edge case. It's about testing the system with expected, valid input.\n\n2. **Verify login with empty password** - This test case covers an edge case because it tests the system's behavior when a required field (password) is left empty. Empty or missing input is an extreme condition that needs to be handled properly.\n\n3. **Verify login with SQL injection** - This test case covers a security edge case or an exceptional scenario where an attacker attempts to inject malicious SQL code. While not a typical input, it's crucial for testing security vulnerabilities.\n\n4. **Verify dashboard loads after login** - Similar to the first test case, this seems to cover a normal path scenario after a successful login. It doesn't specifically target an edge or boundary condition but rather a standard workflow.\n\nBased on the analysis, the test case titles that cover edge cases are:\n\n- 2. Verify login with empty password**
```

Utilities: 19. Get Metadata (Distinct Filter Values)

The screenshot shows the Postman interface for a GET request to `GET 19. Get Metadata (Distinct Filter Values)`. The response is a JSON object with the following fields:

```
{  "success": true,  "metadata": {    "modules": [      "AHC",      "AT(Admission Module)",      "Admin",      "BB(Blood Bank)",      "Digital OP",      "Digital dietician",      "Doctors-Wards",      "Dynamic Forms",      "ES(Appointment)",      "Emergency",      "Human Resources (HR)",      "IP Billing",      "IP Digital",      "IP digital",      "LHK",      "Laboratory",      "MIS",    ]  }}
```

POST 14. Analyze () POST 15. Summariz POST 16. Summariz POST 17. Deduplic POST 18. Test AI Pro GET 19. Get Metadata (Distinct Filter Values)

GET {{baseUrl}} /api/metadata/distinct

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

Body Cookies Headers 8 Test Results 200 OK 5.12 s 793 B Save Response

```
{
  "ward": "Ward-Nursing",
  "ward": "Wards-Doctors",
  "ward": "Wards-MR0",
  "priorities": [
    "P1",
    "P2",
    "P3",
    "P4"
  ],
  "risks": [
    "Critical",
    "High",
    "Low",
    "Medium"
  ],
  "types": [
    "Automated",
    "Manual"
  ]
}
```

Connect Git Console Terminal 0 Import Complete

Globals Vault Tools

20. Get Latest Test Case ID:

POST 14. Analy POST 15. Summ POST 16. Summa POST 17. Dedup POST 18. Test AI GET 19. Get Meta GET 20. Get Latest

GET {{baseUrl}} /api/testcases/latest-id

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

Body Cookies Headers 8 Test Results 200 OK 6.33 s 362 B Save Response

```
{
  "success": true,
  "latestId": 6364,
  "nextId": 6365,
  "nextTestCaseId": "TC_6365",
  "totalTestCases": 4660
}
```

21. Get Environment Variables

The screenshot shows a Postman interface for a GET request to `{{baseUri}}/api/env`. The request is successful with a 200 OK status. The response body is a JSON object containing the following environment variables:

```
1 {
2   "MONGODB_URI": "mongodb+srv://srinikilsrinivasan_db_user:81oJe6Hbzqqg3ff@cluster0.naqrzz.mongodb.net/?
3     appName=Cluster0",
4   "DB_NAME": "Gen_AI_Postman_Testing",
5   "COLLECTION_NAME": "Test_cases",
6   "VECTOR_INDEX_NAME": "test_cases_vector_index",
7   "BM25_INDEX_NAME": "testcases_bm25",
8   "USER_STORIES_COLLECTION_NAME": "User_Stories_Collection",
9   "USER_STORIES_VECTOR_INDEX_NAME": "user_stories_vector_index",
10  "USER_STORIES_BM25_INDEX_NAME": "bm25_user_stories",
11  "MISTRAL_API_KEY": "KwT154ltxAQcZb4fvmvegdV6f0xxmSg",
12  "MISTRAL_EMBEDDING_MODEL": "mistral-embed",
13  "GROQ_API_KEY": "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",
14  "GROQ_RERANK_MODEL": "meta-llama/llama-4-scout-17b-16e-instruct",
15  "GROQ_SUMMARIZATION_MODEL": "openai/gpt-oss-120b"
16 }
```

00. Health Check:

The screenshot shows a Postman interface for a GET request to `{{baseUri}}/api/health`. The request is successful with a 200 OK status. The response body is a JSON object indicating the server is running:

```
1 {
2   "status": "OK",
3   "message": "Server is running"
4 }
```